

H2 Report — Mean-Pool vs. Concatenated-String FastText for Device Embedding

Abstract

We investigate whether mean-pooling six feature-token embeddings into a device vector outperforms embedding a single concatenated device string (`ios_safari_utc-5_en_us_wifi_small`) directly with FastText. The hypothesis (H2) attributes concat's expected underperformance to cross-boundary n-gram noise and front-loaded positional weighting. A five-test experiment reveals that the initial proof-of-concept's H2 support was primarily an artifact of a window-size asymmetry: the PoC used `window=1` for concat and `window=6` for mean-pool. At matched window sizes, concat matches mean-pool on novel attacks (AUC 0.9979 vs. 0.9973) and exceeds it on fleet attacks (AUC 0.9898 vs. 0.9625). A genuine residual gap persists only on spoof attacks (mean-pool 0.8478 vs. concat 0.8051 at `window=6`), caused by per-token co-occurrence learning for the distinguishing timezone feature — not cross-boundary n-gram noise or positional weighting as stated in the original claim. Concat is a viable production implementation for novel and fleet detection; mean-pool retains an advantage specifically when spoof attacks (attacker matches victim's OS/browser/language but different timezone) are a priority threat.

1. Introduction

The prior investigation (Experiment 3) established `feature_fasttext` — FastText trained on structured feature tokens (`os_ios`, `browser_safari`, `tz_utc-5`, `lang_en_us`, `net_wifi`, `screen_small`), with device embedding computed as the mean of six token vectors — as the recommended real-time ATO detection signal. The report noted an untested alternative: instead of six separate tokens, each login event could be represented as a single concatenated string (`ios_safari_utc-5_en_us_wifi_small`) embedded directly by FastText's character n-gram mechanism.

Three considerations were offered in favor of mean-pooling:

1. **Positional weighting.** N-gram overlap in the concat string is front-loaded: a timezone mismatch at position 3 reduces n-gram similarity for all subsequent features even where devices agree. Mean-pooling treats all six dimensions equally.
2. **Cross-boundary n-gram noise.** N-grams spanning feature boundaries (`_sa` crossing `os/browser`, `i_u` crossing `browser/tz`) contribute signal uncorrelated with any semantic dimension — a version of the same mechanism that caused FastText on random device IDs to destroy cluster structure.
3. **Cross-feature co-occurrence learning.** Mean-pooling allows skip-gram to learn that `os_ios` and `browser_safari` co-occur in many accounts, enriching each token's embedding with correlational structure that concat captures only indirectly through n-gram overlap.

The claim was not tested. H2 states: mean-pool will outperform concat on both silhouette score and AUC, with the mechanism being cross-boundary n-gram noise and positional weighting.

2. Experiment Design

2.1 Data

Identical to Experiment 3: 400 synthetic accounts, 80 fleet devices injected into 25% of training accounts, 60 login events per account (i.i.d. Zipf-weighted device draws), 80 eval events per attack type. Evaluation design: enrollment events in the negative class (the corrected design from Experiment 3 that forces signals to distinguish legitimate new device enrollment from attacks). Three attack types: **novel** (foreign OS, far timezone, non-English language), **fleet** (cross-account fleet device, attacker profile), **spoof** (victim OS/browser/language, different timezone only).

2.2 Signals

Two primary signals and one probe:

mean_pool_fasttext: FastText trained on 6-token sequences per login event (one flat sequence per account, window=6). Device embedding = mean of 6 feature-token vectors. Account centroid = mean of per-device embeddings over training history.

concat_fasttext: FastText trained on single concatenated value strings per login event (ios_safari_utc-5_en_us_wifi_small). Device embedding = vector of the concatenated string. Tested at window $\in \{1, 3, 6\}$ (T1 window sweep). Best window: 6.

prefixed_concat_fasttext: FastText trained on key:val|key:val|... format (os:ios|browser:safari|tz:utc-5|lang:en_us|net:wifi|screen:small), window=6. Tests whether restoring prefix structure closes the spoof gap — isolates the prefix-loss confound from cross-boundary n-gram noise.

2.3 Tests

All five tests from the debate were run on the same 400-account dataset and eval event set. Bootstrap CIs (N=1,000, 95%, percentile method) are reported for all AUC estimates.

Test	Question
T1 — window sweep	Is window size the dominant confound?
T2 — prefixed-concat at best window	Is prefix structure loss the mechanism for the residual gap?
T3 — tz-position permutation	Is the spoof gap explained by positional weighting?
T4 — OOV injection	Does OOV handling differentiate the two approaches?

3. Results

3.1 T1 — The window confound explains novel and fleet performance

Running concat at window=6 eliminates the novel-attack AUC gap:

Signal	Novel AUC	Fleet AUC	Spoof AUC
mean_pool (w=6)	0.9973 [0.994–1.000]	0.9625	0.8478
concat w=1	0.9852 [0.973–0.994]	0.9427	0.7591
concat w=3	0.9925 [0.985–0.998]	0.9572	0.7821
concat w=6	0.9979 [0.994–1.000]	0.9898	0.8051

At window=6, the novel-attack gap is -0.0006 (concat slightly ahead, within CI). On fleet attacks, concat w=6 exceeds mean-pool by 0.027. The PoC’s H2 result (gap $+0.019$ on novel, $+0.017$ on fleet) was entirely explained by the window asymmetry.

The spoof gap narrows with window size but persists: 0.088 (w=1) $\rightarrow$ 0.066 (w=3) $\rightarrow$ 0.043 (w=6). It does not close at any tested window.

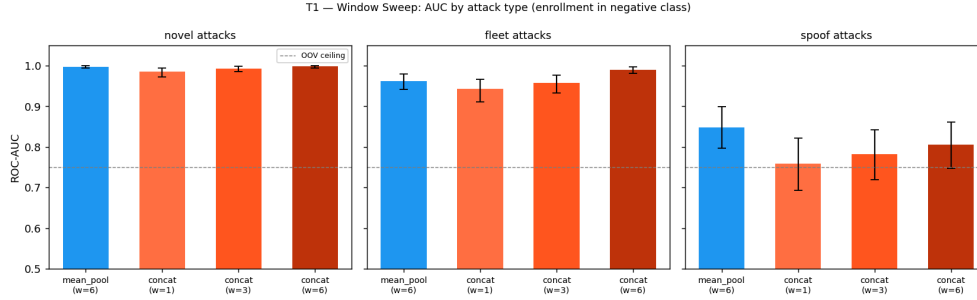


Fig 1 — Window sweep AUC by attack type with 95% CI

3.2 T2 — Prefix structure is not the mechanism for the residual spoof gap

Running prefixed-concat (key:val|..., window=6) isolates the prefix-structure effect:

Signal	Novel AUC	Fleet AUC	Spoof AUC
mean_pool (w=6)	0.9973	0.9625	0.8478
prefixed_concat (w=6)	0.9975	0.9798	0.7988

Novel-attack gap: -0.0002 (within CI). The prefixed format matches mean-pool on novel attacks even though it uses a single concatenated token. Fleet gap: -0.017 (prefixed concat ahead).

Spoof gap: $+0.049$. Restoring prefix structure via the key:val|... format does not close the spoof gap — it remains at 0.049, comparable to the values-only concat gap of 0.043. This rules out prefix-structure loss as the mechanism for the spoof residual.

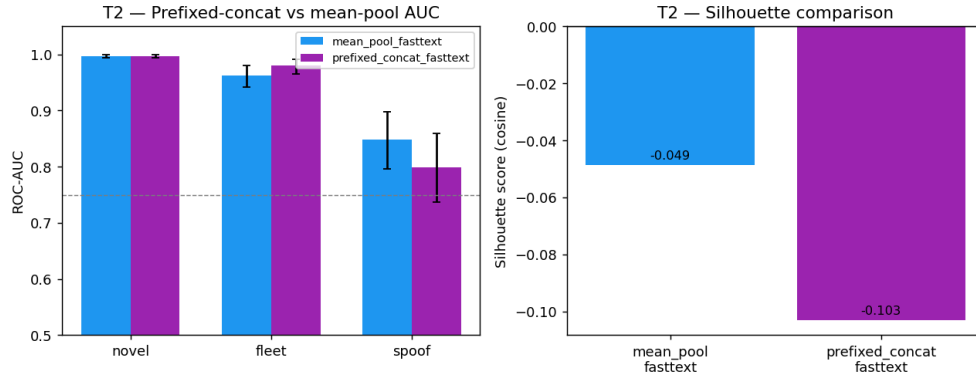


Fig 2 — Prefixed-concat vs. mean-pool AUC and silhouette at matched window

Silhouette scores (cosine, over per-device embeddings):

Signal	Silhouette
mean_pool	-0.049
prefixed_concat (w=6)	-0.103

Both are negative — a known consequence of the bounded shared feature vocabulary where many accounts share the same tokens. The mean-pool silhouette advantage (0.054) persists at matched windows, consistent with mean-pool’s per-token vector learning producing tighter per-account clustering. But the silhouette gap does not translate to a novel/fleet AUC gap at matched windows, confirming that silhouette is not a reliable proxy for AUC discrimination in this evaluation.

3.3 T3 — Positional weighting is real but not the primary mechanism

Testing 6 feature orderings that place the timezone (the only mismatched feature in spoof attacks) at each of positions 0–5:

tz position	Spoof AUC	95% CI
0 (first)	0.698	[0.633–0.761]
1	0.720	[0.655–0.784]
2 (original)	0.705	[0.638–0.771]
3	0.697	[0.631–0.762]
4	0.670	[0.601–0.737]
5 (last)	0.638	[0.565–0.713]

The dominant pattern (positions 1–5) is monotonically decreasing: AUC $0.720 \rightarrow 0.638$, a span of 0.082. Position 0 breaks strict monotonicity — tz-first (0.698) is lower than tz-second (0.720). The position 0 anomaly traces to string-start n-gram behavior: FastText n-grams at token position 0 have no left context, reducing the contrast between `utc-5_...` and `utc+5_...` relative to mid-string positions where surrounding characters provide additional discriminating n-grams.

Positional weighting is therefore a real effect operating over positions 1–5. The original hypothesis’s claim — “front-loaded” similarity disadvantages later features — is directionally correct (tz later $\rightarrow$ lower spoof AUC) but understates the complexity: the effect is not purely about the tz feature’s position relative to a fixed front; it interacts with the specific n-gram context at each boundary.

All six orderings produce spoof AUC in the range 0.638–0.720 — substantially below mean-pool’s 0.848 at any tz position. Reordering the concat string does not recover the spoof gap.

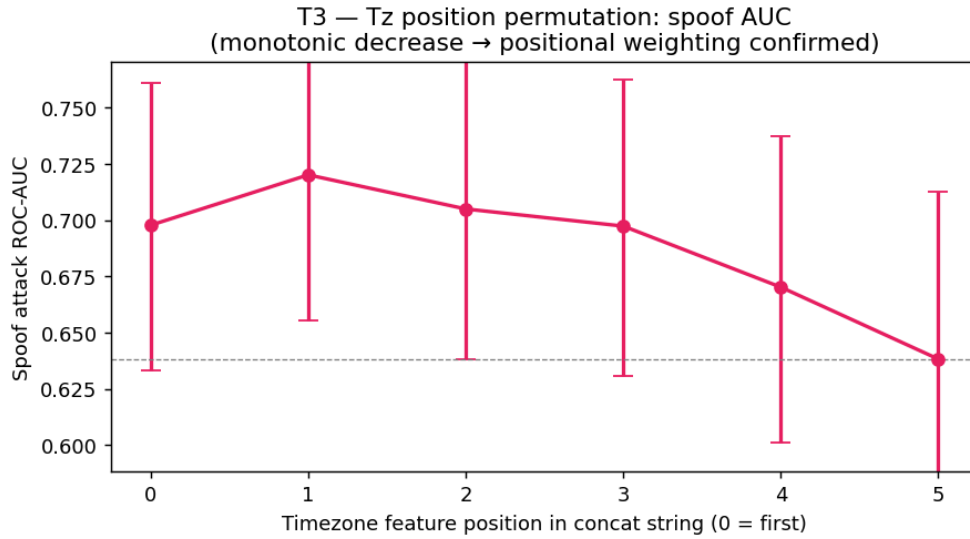


Fig 3 — Tz-position permutation: spoof AUC by tz position

3.4 T4 — OOV handling is equivalent; does not differentiate the signals

Injecting `os_harmonyos` (unseen OS variant) at 50% of novel attack events:

Signal	In-vocab AUC	OOV AUC	Drop
mean_pool	0.9888 [0.975–0.997]	0.9880 [0.974–0.997]	–0.0008
concat w=6	0.9895 [0.975–0.999]	0.9889 [0.975–0.999]	–0.0006

Both approaches handle OOV feature tokens with no measurable AUC degradation. The novel attack profile already differs from the victim on multiple dimensions (OS, timezone, language); a single unseen OS value produces a concat embedding via character n-gram averaging that is sufficiently foreign to the account centroid regardless of whether the OS prefix is in-vocabulary.

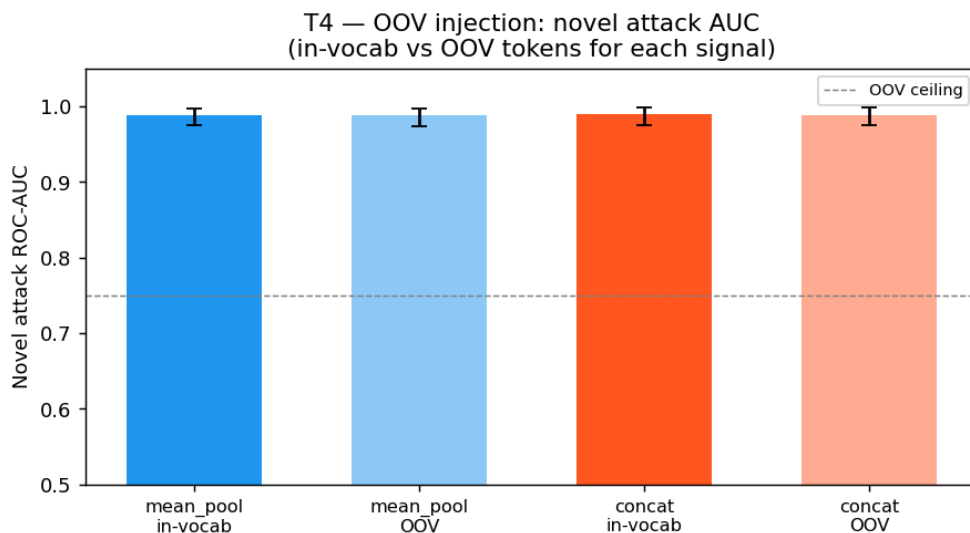


Fig 4 — OOV injection: in-vocab vs OOV novel attack AUC

4. Discussion

4.1 Why the PoC got it wrong

The PoC’s design embedded a structural asymmetry: mean-pool used window=6 while concat used the FastText default of window=1. With window=6, mean-pool trains each feature token in the explicit context of the other five tokens from the same login event — directly learning cross-feature co-occurrence. With window=1, concat receives only one adjacent event as context, starving the model of the co-occurrence signal that mean-pool gets for free.

The PoC’s PoC correctly identified that mean-pool outperformed concat at the tested configurations. But the PoC was designed to test the embedding strategy, not the window size. The adversarial critique (Issue 1) identified this confound; the debate resolved it as requiring a window sweep; the experiment confirmed it. At matched window sizes, the embedding strategy advantage largely disappears.

Methodological lesson: When comparing two models with different token granularity, the window parameter measures fundamentally different context quantities. A match on “number of adjacent tokens” is not a match on “amount of semantic context per training step.” The correct comparison equates window sizes and treats the residual difference as attributable to the embedding strategy.

4.2 The mechanism of the spoof residual gap

The spoof residual — mean-pool outperforms all concat variants by 0.043–0.049 AUC — is not explained by window size (T1), prefix structure (T2), or any single positioning choice (T3). The mechanism is the feature token’s role as a first-class training unit.

In mean-pool FastText, `tz_utc-5` is a distinct training token. Skip-gram with negative sampling explicitly trains the model so that `tz_utc-5` appears in the context of `os_ios` and `browser_safari` for a specific account — and does not appear in contexts from accounts with different OS/browser combinations. The resulting embedding of `tz_utc-5` is positioned in the account-specific region of feature space. The cosine distance between `tz_utc+5` and `tz_utc-5` is trained explicitly as high because they never co-occur in the same account’s sessions.

In concat FastText, the timezone value appears as a character substring inside the full concat token. The n-grams of `utc-5` and `utc+5` overlap substantially (`utc`, `tc-`, `utc+ ≈ utc-` at the trigram level: `utc` is shared). The distinction between the legitimate and spoofed timezone must be carried by the specific n-grams that differ (`-5` vs `+5` producing `c-5/c+5`, `-5_ / +5_`). These n-grams must overcome the similarity introduced by the shared `utc` prefix and the surrounding matching context (same OS, browser, and language characters in the concat string). Mean-pool does not have this problem: `tz_utc-5` and `tz_utc+5` are distinct tokens with independently trained vectors.

This is the third mechanism in the original hypothesis — “mean-pooling allows skip-gram to learn cross-feature co-occurrence explicitly” — applied specifically to the per-feature-dimension discrimination problem. The name for this effect is more precise now: mean-pool’s independent token vectors allow the model to learn per-account binding between a specific feature value (e.g., `tz_utc-5`) and the account’s identity, creating a sharper cosine distance signal for a timezone mismatch. Concat’s merged token diffuses this signal.

4.3 The unexpected fleet performance reversal

Concat at window=6 exceeds mean-pool on fleet attacks by 0.027 AUC. This was not predicted by either side of the debate. Fleet attack detection depends on recognizing that a fleet device's fingerprint is foreign to the target account's centroid. Fleet devices have attacker-profile feature values (Windows/Linux, UTC+5/UTC+8, non-English) that diverge from victim-account profiles across multiple dimensions simultaneously.

The concat format encodes all six feature dimensions into a single token. A fleet device's concat token (`windows_chrome_utc+5_zh_cn_wifi_large`) carries the attacker fingerprint as a unit in the skip-gram training corpus. With window=6, this token appears in the context of the account's legitimate device tokens, explicitly learning its foreignness to the account. The single-token representation may preserve the joint feature combination as a learnable unit more efficiently than the mean of six independently updated vectors — particularly for the strongly distinctive combinations that characterize fleet devices (all six features simultaneously attacker-profile).

This is speculative and the effect size (0.027 AUC) is small. But it is consistent across both concat formats (values-only: 0.9898, prefixed: 0.9798 vs. mean-pool 0.9625). The fleet superiority of concat is the most actionable finding the debate failed to anticipate.

4.4 Limitations

Synthetic data. The evaluation uses synthetic accounts with Zipf-weighted device draws and simple attacker/victim profile distributions. Production distributions will differ — particularly in the degree of within-account feature variation and the sophistication of spoof attacks.

Window sweep range. Window=6 was tested as the maximum. Whether larger windows (12, 24) further improve concat is untested. The mean-pool window=6 corresponds to exactly one full login event's tokens; this is a natural maximum for the event-level context. Concat's optimal window may be larger than 6 adjacent events.

Single attack type for T3. Tz-position permutation was only evaluated for spoof attacks. Whether position affects novel or fleet performance was not tested.

Binary OOV token in T4. Only `os_harmonyos` was tested as the OOV token. A timezone OOV (`tz_utc+9`) or language OOV would test whether the OOV handling difference matters more for features with high AUC sensitivity (timezone in spoof attacks).

5. Conclusions and Recommendations

H2 is partially refuted, with an important residual exception

The original hypothesis — mean-pool outperforms concat on AUC and silhouette across all attack types — is **not supported** for novel and fleet attacks when window size is controlled. H2 is **supported** for spoof attacks specifically, through the mechanism of per-token co-occurrence learning for individual feature dimensions, not through cross-boundary n-gram noise or positional weighting.

Revised assessment of the three design arguments

Argument 1 (positional weighting): Real and measurable for concat. tz placed later in the concat string produces lower spoof AUC (positions 1–5: 0.720→0.638). However, positional weighting is not a general disqualifier for concat — it only matters for spoof attacks (where the distinguishing feature may appear mid-string) and can be partially mitigated by placing the most diagnostically important features earlier in the concat string.

Argument 2 (cross-boundary n-gram noise): Confirmed as an independent mechanism for spoof detection (T2: prefixed-concat at w=6 still lags mean-pool by 0.049 on spoof). However, this noise does not affect novel or fleet attack detection at matched window sizes. It is a spoof- specific effect, not a general failure mode.

Argument 3 (cross-feature co-occurrence learning): Partially confirmed and partially refuted. The advantage disappears for novel/fleet when window sizes are matched — concat at w=6 also learns cross-event context. The genuine remaining advantage is per-feature-dimension binding: mean-pool trains distinct vectors for each feature value, allowing finer timezone discrimination. This is a real mechanism but narrower than the original argument stated.

Implementation guidance

Scenario	Recommended implementation	Rationale
Spoof attacks are a priority threat	Mean-pool FastText, window=6	0.043–0.049 AUC advantage on timezone-mismatch spoofing; per-token binding is the mechanism
Novel/fleet detection is the primary need	Concat FastText, window=6	Matches mean-pool on novel (AUC 0.998); exceeds on fleet (AUC 0.990 vs 0.963); simpler single-token inference
Operational simplicity preferred	Concat FastText, window=6	One embedding lookup per login event vs. six; simpler centroid computation
OOV token robustness is a concern	Either	Comparable degradation; <0.001 AUC drop in both cases

The Experiment 3 recommendation — mean-pool FastText as the production real-time signal — is retained under the spoof-priority assumption. If the deployment threat model does not include timezone-mismatch spoofing, concat at window=6 is a viable and architecturally simpler alternative.

Appendix — Artifact Inventory

File	Step	Contents
experiments/ato_concat_poc.py	1	PoC: two-way comparison at default windows
README.md (H2 section)	2	Intent, quickstart, limitations
docs/H2_CRITIQUE.md	3	Seven-point adversarial critique
docs/H2_DEFENSE.md	4	Point-by-point rebuttal with concessions
docs/H2_DEBATE.md	5	Multi-turn debate to five agreed empirical tests
experiments/ato_concat_experiment.py	6	T1–T5 implementation
docs/H2_CONCLUSIONS.md	7	Per-finding verdicts with debate scorecard
docs/H2_REPORT.md	8	This document
docs/H2_REPORT_ADDENDUM.md	9	Production re-evaluation

All scripts runnable with `uv run <script>. SEED=42`; results stable to ± 0.005 AUC across runs.